

MC809 Computação Distribuída

Laboratório 4

Nome: Arthur Costa Barreto

RA: 206325

Data: Setembro de 2026

1. Introdução

1.1. Descrição da Proposta do Laboratório

Este laboratório se trata de uma exploração de um modelo utilizado para reconhecimento de similaridade entre imagens, denominado de rede siamesa. Com esse objetivo, utilizaremos o dataset “fashion mnist” para sortear duas imagens aleatoriamente e passá-las individualmente por duas redes distintas que compartilham a mesma arquitetura e os mesmos pesos.

Para integrar este laboratório com o conteúdo de computação distribuída, as duas redes serão simuladas como se estivessem em clientes distintos e o vetor de saída da rede ao receber uma imagem como entrada será enviado para um servidor. O servidor, por sua vez, recebe os dois vetores de saída e realiza o cálculo da distância euclidiana entre eles. Se a distância for maior do que um certo threshold escolhido, o modelo classifica as imagens como pertencentes a classes distintas.

1.2. Divisão dos arquivos

Para este exercício, existem 4 arquivos principais:

- `buil_base_model.ipynb`: Notebook no qual a rede siamesa é treinada e os resultados são exibidos numericamente e na forma de plots;
- [`client.py`](#): Arquivo que contém a definição da classe cliente e que envia o output do modelo para o servidor;
- [`server.py`](#): Arquivo que contém a definição da classe servidor e calcula a distância euclidiana entre os vetores de saída dos modelos enviados pelos clientes. Em seguida, exibe no terminal se os vetores são gerados por imagens de mesma classe ou não.
- `data_payload.proto`: Arquivo que contém a estrutura da mensagem comunicada entre cliente e servidor. Para este laboratório, ela carrega `repeated double` (vetor de saída), `pair_id` (identifica qual o par de cada imagem), `client_id` e `class_id` (rótulo da imagem).

2. Construção e treinamento da rede siamesa

2.1 Estratégia de treinamento

O treinamento foi dividido em 2 etapas. A primeira etapa é o treinamento de uma rede convolucional classificadora, que é responsável apenas por inicializar os pesos da futura rede siamesa. Nesta etapa, adiciona-se uma camada extra de 10 neurônios que recebe como entrada a saída vetorizada - que na segunda etapa é enviada para a rede siamesa. Acredita-se

que essa inicialização ajuda a rede siamesa, pois torna o modelo capaz de separar imagens de classes distintas.

A segunda etapa de treinamento envolve captar a saída das camadas convolucionais antes do colapso em 10 neurônios. No modelo proposto, que será melhor explicado na seção seguinte, a saída é emitida após passar por uma camada de 128 neurônios. Essa saída é, então, utilizada para realizar apenas a operação de distância euclidiana e comparar o resultado a um threshold.

Para ajustar os pesos do modelo, utilizou-se a perda contrastiva, dada por:

$$L = (1 - y) \cdot D^2 + y \cdot \max(0, m - D)^2$$

Onde, m é um hiperparâmetro de margem definido em 1.0 em meu setup. Além disso, o otimizador escolhido foi o AdamW com `learning_rate` de $1e-4$: `learning_rate` mais baixo para evitar o colapso das camadas extratoras de features iniciais pré inicializadas. Definiu-se, ainda, um batch de 64 pares de imagens treinadas por 10 épocas.

Vale mencionar que apesar de serem estágios diferentes, os pesos sendo treinados em ambas etapas é o mesmo. A única diferença é que a saída passa a ser dada pela distância euclidiana entre os vetores de saída das camadas iniciais do modelo para cada imagem no par de entrada. Logo, ajusta-se os mesmos pesos do estágio 1, mas a partir de uma loss diferente e um `learning_rate` menor, considerando pesos bem inicializados.

2.2 Preparação dos dados

Para o treinamento do primeiro estágio classificador, o pré-processamento dos dados é simples, já que utilizamos o dataset importado da lib do tensorflow. A única operação feita sobre os dados é a divisão do valor dos pixels por 255 para manter todos os valores de entrada entre 0 e 1, auxiliando, assim, o processo de treinamento do modelo ao evitar problemas no cálculo e propagação dos gradientes.

Para o segundo estágio, além do que já foi feito no primeiro, também é necessário agrupar os dados de entrada em pares. Dessa forma, a cada etapa do treinamento o modelo irá receber duas imagens distintas, que passam pelo mesmo backbone convolucional, e gerar uma saída baseada na distância calculada entre os vetores que saem da camada densa. Uma preocupação que surgiu dessa etapa foi o desbalanço entre duplas de imagens da mesma classe e duplas de entradas de classes distintas. Se o sorteio fosse completamente aleatório, grande parte do novo dataset seria de imagens que pertencem a classes diferentes, pois o dataset original possui 10 classes praticamente igualmente distribuídas.

Portanto, para evitar o colapso do modelo, o rótulo do par é sorteado antes da segunda imagem: decide-se com probabilidade $1/2$ se o par será positivo ou negativo e só então sorteia-se a segunda imagem dentro da classe da âncora (positivo) ou em uma classe deslocada aleatoriamente (negativo). Isso garante $\sim 50\%$ de cada tipo no conjunto de 60000 pares, gerado uma única vez antes do treino.

2.3 Construção do modelo

A rede foi dividida em 2 estágios: classificador convolucional e camada siamesa. O primeiro possui 2 camadas convolucionais intercaladas por Batch Normalization e MaxPooling2D. Logo após a segunda camada convolucional, há um Flatten para permitir a

saída vetorizada e, por fim, uma camada densa de 128 neurônios seguida por UnitNormalization. Enquanto isso, o segundo estágio pega a saída vetorizada e normalizada de cada imagem que passa pelo primeiro estágio e realiza o cálculo da distância euclidiana. Uma imagem completa da rede siamesa proposta pode ser vista a seguir:

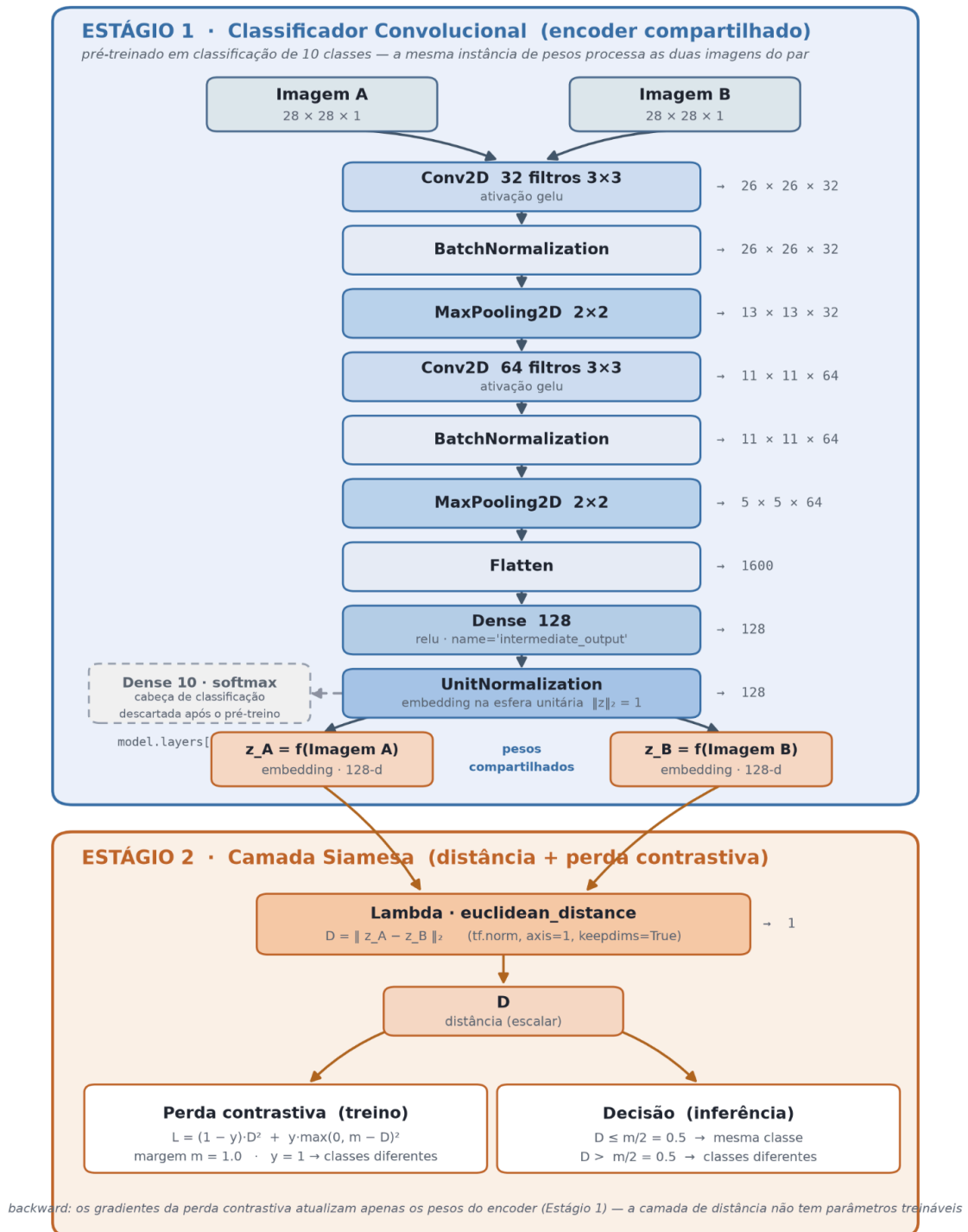


Imagem 1: Diagrama da rede siamesa

2.4 Resultados de treinamento da rede siamesa

Durante o treinamento, podemos verificar uma acurácia de validação superior a 94,5%, acompanhando de perto o valor da acurácia de treino, que foi de 95,5%. Pelos plots, podemos verificar que houve um overfit bem leve ao treinar por 10 épocas, mas nada acima do normal já que a acurácia de validação ainda subia mesmo após as 10 épocas.

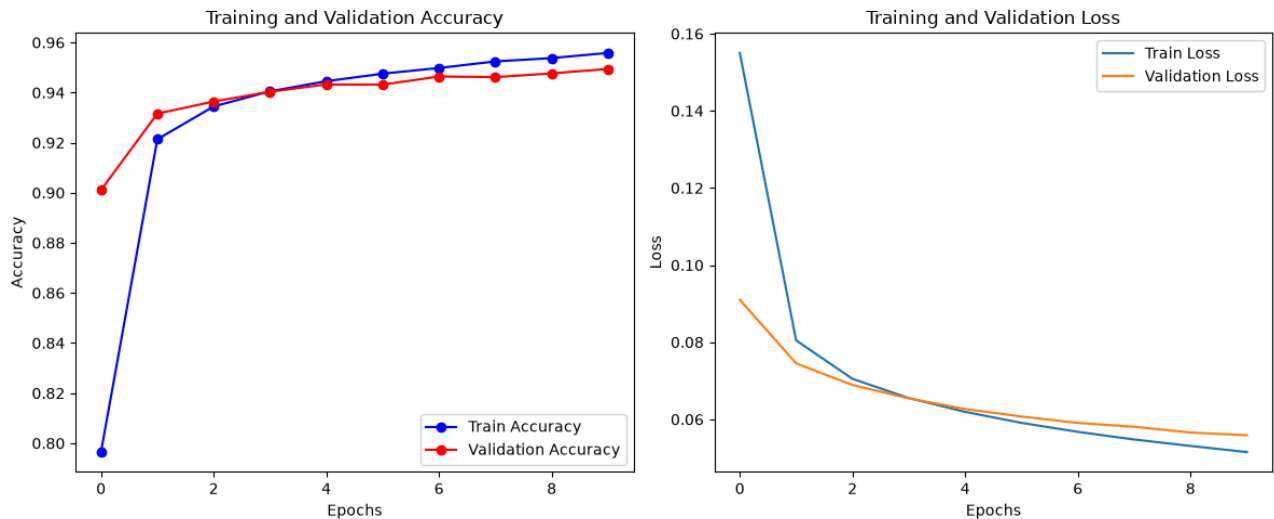


Imagem 2: Gráficos de acurácia e perda vs número de épocas

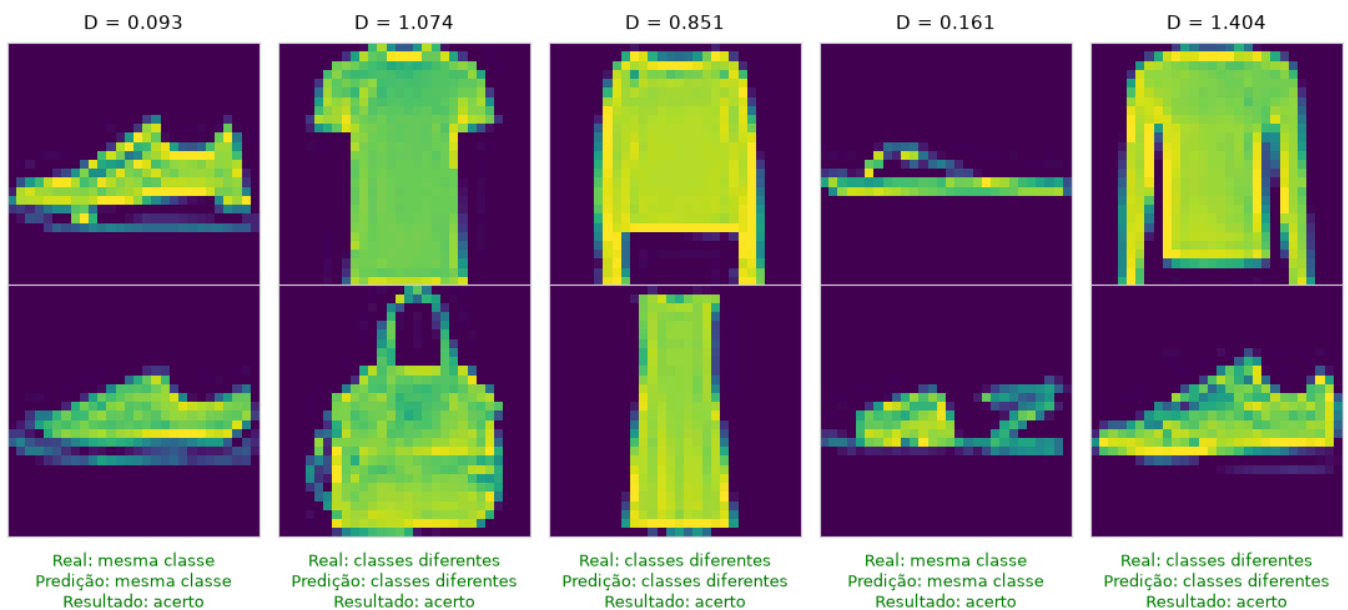


Imagem 3: Resultado de classificação da rede siamesa para amostras aleatórias

Para o dataset aleatório gerado, a rede siamesa atingiu acurácia de 93,46%, precisão de 92,38% e recall de 94,84%. Conclui-se, portanto, que o objetivo foi atingido com sucesso na medida em que o modelo é capaz de identificar imagens de classes distintas com boa acurácia.

Obs.: Os resultados exatos podem não ser obtidos ao rodar o mesmo notebook, pois o dataset é gerado aleatoriamente.

3. Dinâmica cliente servidor

3.1 Protocolo de comunicação

A comunicação entre os nós é feita com gRPC, e o contrato está declarado em `data_payload.proto` (proto3, pacote federated). O serviço expõe um único RPC unário:

```
service SimilarityService {  
  rpc CompareData (DataPayload) returns (google.protobuf.Empty);  
}
```

A mensagem `DataPayload` carrega `output_vector` (repeated double, os 128 valores do embedding), `pair_id`, `client_id` e `class_id`. A partir desse arquivo são gerados os stubs usados pelos dois lados.

Vale destacar que o RPC retorna `google.protobuf.Empty`: o cliente não espera o resultado, que aparece apenas no log do servidor. Uma alternativa seria devolver a distância e a decisão na própria resposta, mas isso obrigaria o servidor a responder ao primeiro cliente antes de conhecer o segundo vetor.

3.2 Cliente

Toda a lógica de exploração e treinamento da rede siamesa é feita no notebook `build_base_model.ipynb`, como descrito na seção 1.2. Logo, o script `client.py` apenas carrega o modelo pré-treinado (`fashion_mnist_model.keras`, o estágio 1 já truncado na saída de 128 dimensões) e sorteia aleatoriamente pares de imagens do Fashion-MNIST, limitado ao número de observações no hiperparâmetro `SAMPLES`.

A classe `Client` encapsula o canal (`grpc.insecure_channel("localhost:50051")`), o stub `SimilarityServiceStub` e o `client_id`. O método `forward()`, definido no arquivo `client.py`, executa o modelo base com `training=False` — importante porque a rede tem camadas de Batch Normalization, que se comportam de forma diferente em treino e inferência — e `send_output_vector()`, que achata o tensor resultante (`.numpy().ravel().tolist()`), monta o `DataPayload` e faz a chamada `CompareData`.

No experimento, duas instâncias de `Client` (com `client_id` 1 e 2) simulam nós distintos: o que muda entre eles é apenas a imagem processada e o identificador, já que ambos compartilham a mesma arquitetura e os mesmos pesos, conforme exige a rede siamesa. Cada iteração do laço define `pair_id = i`, que é o identificador usado pelo servidor para casar as duas mensagens de uma mesma amostra.

3.3 Servidor

O arquivo `server.py` sobe um `grpc.server` com `ThreadPoolExecutor(max_workers=5)` escutando em `[::]:50051` e carrega o segundo estágio da rede (`fashion_mnist_siamese_model.keras`). Esse carregamento depende de um quinto arquivo, `keras_distance.py`: a camada `Lambda` que calcula a distância só pode ser desserializada porque a função `euclidean_distance` está registrada com `@keras.saving.register_keras_serializable` e é passada em `custom_objects`. Usar o próprio

modelo salvo — em vez de recalcular a norma em NumPy — garante que a operação executada em produção é exatamente a mesma usada durante o treino.

O ponto central do servidor é o encontro dos dois vetores, já que cada cliente chega em uma mensagem independente e, potencialmente, em threads diferentes do pool. Cada chamada armazena o vetor em `self.pending[pair_id][client_id]`, sob um `threading.Lock()` que protege o dicionário de acessos concorrentes. Enquanto o par tiver menos que `EXPECTED_CLIENTS = 2` entradas, o servidor apenas responde `Empty`; quando o segundo vetor chega, o par é removido da estrutura com `pop` e só então a distância é calculada.

A decisão final compara a distância com `THRESHOLD = 0.5`, que não é um valor arbitrário: corresponde a $m/2$, o ponto médio da margem $m = 1.0$ usada na perda contrastiva do treino, o mesmo critério aplicado na avaliação da seção 2.4. Apenas a fins de demonstração, cada cliente também envia o rótulo verdadeiro da sua imagem (`class_id`); o servidor compara os dois rótulos e registra em log se a resposta do modelo foi correta, sem que essa informação participe da decisão.

3.3 Resultados parciais

A demonstração é executada em dois terminais, primeiro `python server.py` e depois `python client.py`. A saída do servidor para `SAMPLES = 5` foi:

```
2026-09-25 23:20:54 Server started at 50051
2026-09-25 23:20:57 Clients 1 and 2 are not similar (distance: 1.0432).
2026-09-25 23:20:57 Correct answer is: not similar
2026-09-25 23:20:57 Clients 1 and 2 are not similar (distance: 1.4139).
2026-09-25 23:20:57 Correct answer is: not similar
2026-09-25 23:20:57 Clients 1 and 2 are not similar (distance: 0.5138).
2026-09-25 23:20:57 Correct answer is: not similar
2026-09-25 23:20:57 Clients 1 and 2 are not similar (distance: 1.4066).
2026-09-25 23:20:57 Correct answer is: not similar
2026-09-25 23:20:57 Clients 1 and 2 are not similar (distance: 1.4127).
2026-09-25 23:20:57 Correct answer is: not similar
```

As cinco amostras foram classificadas corretamente. Note, porém, que o sorteio do `client.py` é uniforme, sem o balanceamento aplicado no treino — como o dataset tem 10 classes, cerca de 90% dos pares sorteados caem em classes diferentes, e foi exatamente o que ocorreu nesta execução. Chama atenção ainda o terceiro par, com distância 0,5138: correto, mas muito próximo do limiar, o que ilustra a região de incerteza do modelo.

4. Conclusão

Dos resultados apresentados na seção 2.4 é possível concluir que o modelo se comporta da forma esperada e o objetivo inicial de distinguir imagens de mesma classe foi atingido. No entanto, é preciso tomar muito cuidado durante o treinamento, pois caso o dataset apresente um desbalanço entre os pares de imagens (muitos pares de classe distinta e

poucos de classes iguais), o modelo tenderá a responder apenas o que é prevalente no dataset de treino e sofrerá com dados reais.